

Computer Vision

Lecture 5: Training Deep Networks & CNN Architectures

Nguyen Manh Toan

Swinburne Vietnam

Week 5

Last week built convolutional layer and learned to differentiate it.

$$s = \text{FC}(\text{pool}(\text{conv}(x))), \quad \text{trained by backpropagation}$$

What we can now do

- Stack convolutions to grow the receptive field
- Compute every gradient automatically
- Train a small CNN on a small dataset

What we cannot yet do

- Train anything *deep* — gradients vanish, losses plateau, runs diverge
- Say which of the infinitely many stackings to use

Today: first the techniques that make depth trainable, then the architectures they enabled.

Today's Lecture

- 1 Initialization
- 2 Normalization
- 3 Optimizers
- 4 Augmentation & Regularization
- 5 CNN Architectures
- 6 Transfer Learning
- 7 Wrap-Up

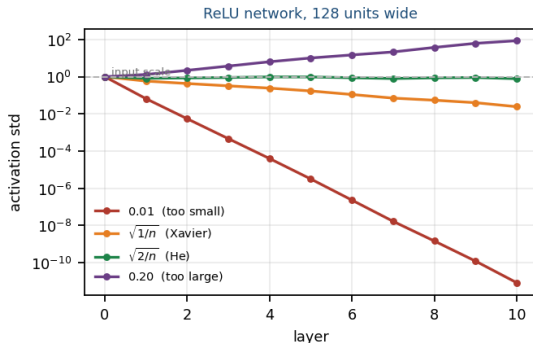
Initialization

Why Not Just Use Small Random Numbers?

All zeros — fatal. Every unit in a layer computes the same thing, receives the same gradient, and stays identical forever. *Symmetry never breaks.*

Small Gaussian, $W \sim 0.01\mathcal{N}(0, 1)$ — fine for a few layers, fatal for many: activations shrink at every layer until they are numerically zero, and so are the gradients.

Large Gaussian, $W \sim \mathcal{N}(0, 1)$ — activations explode, or saturate the nonlinearity.



Activation std through a 10-layer network

The Variance Argument

For one unit, $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$, with w and x independent and zero-mean:

$$\text{Var}(z) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$$

To keep the variance **unchanged** from layer to layer we need

$$n_{\text{in}} \text{Var}(w) = 1 \implies \text{Var}(w) = \frac{1}{n_{\text{in}}}$$

- Too small: variance shrinks by a factor each layer, so it decays *geometrically* with depth
- Too large: it grows geometrically

Xavier / Glorot (2010)

Balancing forward and backward passes gives

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Derived for $\tanh$, symmetric about zero.

He Initialization — the ReLU Correction

ReLU sets half its inputs to zero, so it **halves** the variance of what passes through. Compensate with a factor of two:

$$\boxed{\text{Var}(w) = \frac{2}{n_{\text{in}}}} \quad w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

- He et al., 2015 — the same paper series as ResNet
- Without it, a 30-layer ReLU network barely trains; with it, depth is routine
- The default in PyTorch for Conv2d and Linear

In practice

```
nn.init.kaiming_normal_(w,  
    mode='fan_in',  
    nonlinearity='relu')
```

Biases start at zero. You will rarely change this — but you should recognize the symptom when someone has.

Normalization

Batch Normalization

Good initialization sets the scale at step 0. **Nothing keeps it there.** So normalize explicitly, as a layer, over the batch:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B^{(k)}}{\sqrt{(\sigma_B^{(k)})^2 + \epsilon}}, \quad z^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

- μ_B, σ_B are computed **per channel, across the batch** (and across space, for conv)
- γ, β are **learned** — the network can undo the normalization if it wants to
- Placed between the convolution and the nonlinearity
- It is differentiable, so it trains like anything else

Why γ, β ?

Forcing every activation to zero mean and unit variance would be a constraint, not a help — a sigmoid would be pinned to its linear region. The learned scale and shift give the freedom back.

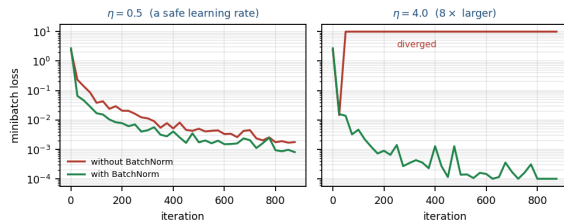
Pros and Cons BatchNorm Buys, and What It Costs

Pros

- Much higher learning rates become stable
- Far less sensitivity to initialization
- Faster convergence — often several times fewer epochs
- A mild regularizing effect: each example is normalized using *whichever* others share its batch

Cons

- Behaves differently at train and test time — a genuine source of bugs
- Breaks down at very small batch sizes



Same network, same learning rate, with and without
BN

Train Time vs. Test Time

Training

Normalize with the statistics of the **current batch**. Meanwhile, keep a running average of μ and σ^2 .

Inference

Use the stored **running averages**. A prediction must not depend on which other images happen to be in the batch.

The bug you will write at least once

Forget `model.eval()` before validating, and BatchNorm keeps using batch statistics — so your validation accuracy depends on batch composition, and a batch of size 1 produces nonsense. `model.train()` to switch back. (Dropout has the same train/test split.)

The Normalization Family

All compute the same $\hat{x} = (x - \mu)/\sigma$; they differ in *which axes* μ and σ are taken over.

- **BatchNorm** — over (N, H, W) , per channel. Depends on batch size
- **LayerNorm** — over (C, H, W) , per example. Batch-independent; standard in transformers
- **InstanceNorm** — over (H, W) , per example per channel. Style transfer
- **GroupNorm** — over channel groups. Works at batch size 2, used in detection

Why it works — unsettled

The original paper argued it reduces “internal covariate shift”. Later work (Santurkar et al., 2018) showed the effect persists when covariate shift is deliberately reintroduced, and argues instead that it **smooths the loss landscape**.

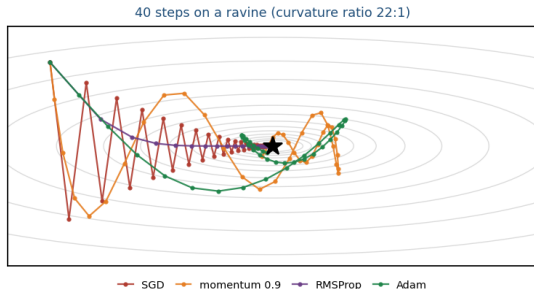
A rare case of a technique whose usefulness is settled and whose explanation is not.

Optimizers

The Trouble With Plain SGD

- **Ravines:** when the loss curves far more steeply in one direction than another, SGD zig-zags across the valley and crawls along it
- **Saddle points:** gradients near zero in every direction, and these dominate in high dimensions — far more common than true local minima
- **One learning rate for every parameter**, however differently they scale

Each fix below targets one of these.



The same ravine, four optimizers

$$v \leftarrow \mu v - \eta \nabla_W L, \quad W \leftarrow W + v, \quad \mu \approx 0.9$$

- v is a **velocity**, not a step: gradients accumulate rather than being applied once
- Consistent directions build speed; oscillating ones cancel — so the zig-zag damps and the valley floor accelerates
- Effective step size approaches $\frac{\eta}{1 - \mu}$, so $\mu = 0.9$ is roughly a 10× amplification
- Carries the optimizer through small saddles and plateaus

Nesterov

Evaluate the gradient *after* the momentum step rather than before — a look-ahead correction. Slightly better in theory and usually in practice; `nesterov=True`.

SGD + momentum 0.9 still trains most vision models today.

Adaptive Learning Rates: RMSProp and Adam

The idea: parameters with consistently large gradients should take smaller steps, and vice versa. Track a running average of the squared gradient:

$$s \leftarrow \beta_2 s + (1 - \beta_2) (\nabla L)^2$$

$$W \leftarrow W - \frac{\eta}{\sqrt{s} + \epsilon} \nabla L$$

That is **RMSProp** — a per-parameter learning rate.

Adam = momentum + RMSProp, with bias correction:

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L$$

$$s \leftarrow \beta_2 s + (1 - \beta_2) (\nabla L)^2$$

$$W \leftarrow W - \eta \frac{\hat{m}}{\sqrt{\hat{s}} + \epsilon}$$

Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$ work startlingly often.

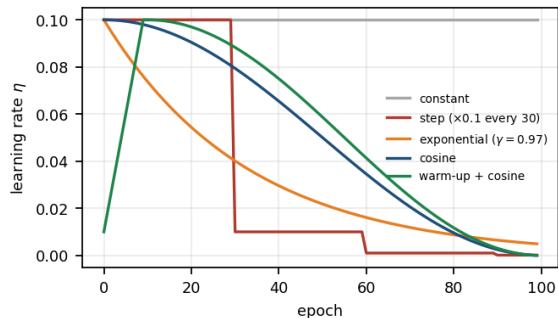
Which one?

Adam to get something working fast, and for transformers and generative models. **SGD + momentum** often generalizes slightly better on vision benchmarks, if you are willing to tune. Start with Adam; switch only if you have a reason.

Learning Rate Schedules

A fixed η is a compromise: large enough to make progress early is too large to settle later.

- **Step:** multiply by 0.1 at fixed epochs — the classic ResNet recipe
- **Cosine:** smooth decay to zero; now the default
- **Exponential:** $\eta_t = \eta_0 \gamma^t$
- **Warm-up:** start small for a few hundred iterations, then rise — essential for large batches and transformers



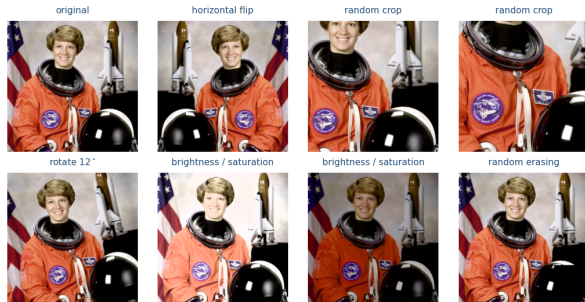
Four schedules over 100 epochs

Augmentation & Regularization

Data Augmentation

Apply label-preserving transformations at training time. The model sees different versions of every image every epoch.

- **Geometric:** random crop, horizontal flip, small rotation, scale
- **Photometric:** brightness, contrast, saturation, hue jitter
- **Erasing:** cutout / random erasing
- **Mixing:** MixUp, CutMix — blend two images *and* their labels



AlexNet's augmentation alone cut its error by over 1% — with $2048\times$ more training data, for free.

One image, eight training views

Choose Augmentations That Preserve the Label

Think before flipping

- Horizontal flip: fine for cats, **wrong** for digits and text
- Vertical flip: wrong for almost every natural photograph
- Heavy color jitter: wrong when color *is* the class: traffic lights, ripe fruit, medical stains

The right question

“Could this transformed image plausibly appear in my test set, with the same label?”

Augmentation encodes the **invariances you believe the task has**. It is domain knowledge, expressed as code — one of the few places your judgement about the data beats a bigger model.

Test-time augmentation: average predictions over several crops or flips. Costs inference time, usually buys 0.5–1%.

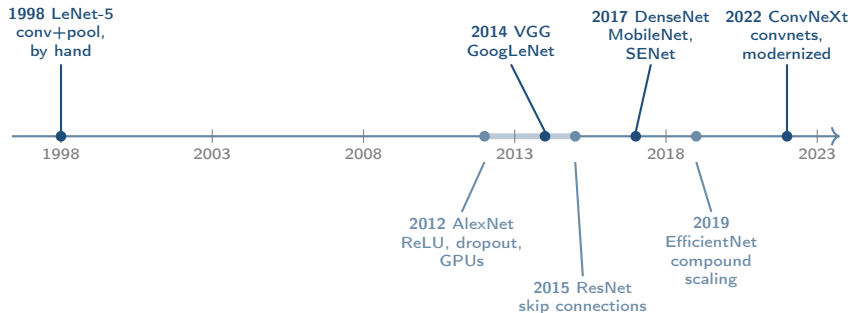
Regularization in Deep Networks

Method	Where	What it does
Weight decay (L_2)	optimizer	Shrinks weights; 10^{-4} is the usual starting point
Dropout	FC layers	Zeroes each unit with probability p at train time; forces redundancy. Largely displaced by BN in conv nets
Data augmentation	input	The most effective regularizer in vision
Early stopping	training loop	Keep the checkpoint with the best validation score
Label smoothing	loss	Target $1 - \epsilon$ instead of 1; discourages over-confidence

If your train-validation gap is large, reach for augmentation first, weight decay second, and a smaller model last.

CNN Architectures

A Timeline of CNN Architectures



- Fourteen years pass between LeNet and AlexNet — the idea was not the bottleneck
- The shaded stretch, **2012–2015**, is where the whole design language was settled
- After ResNet the goal changes: not “deeper”, but **cheaper** and **better-scaled**
- Every model on the right half still uses convolution, BatchNorm, ReLU, and residual connections

LeNet-5 (1998)

- The beginning of the CNN, proposed by Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner
- Designed for handwritten and machine-printed character recognition

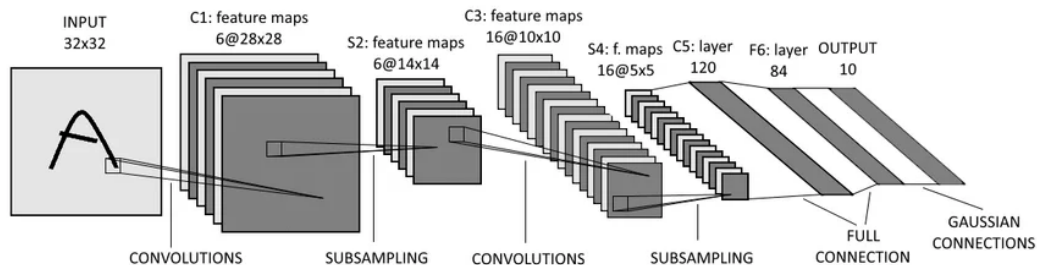


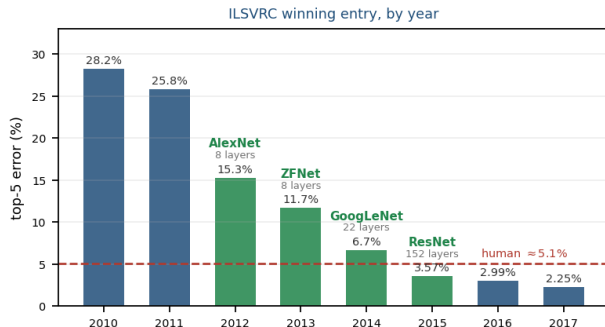
Figure: LeNet-5 architecture: **conv layer:** kernels size 5×5 , stride 1; **avg pooling layer:** kernel size 2×2 , *stride2*; **activation:** tanh.

Eight Years of ImageNet

ILSVRC: 1.28M training images, 1000 classes, scored on **top-5 error**.

- 2011, best classical pipeline: 25.8%
- 2012 **AlexNet**: 15.3% — a 10-point jump, and the end of the argument
- 2014 **GoogLeNet**: 6.7% (VGG: 7.3%)
- 2015 **ResNet**: 3.57%

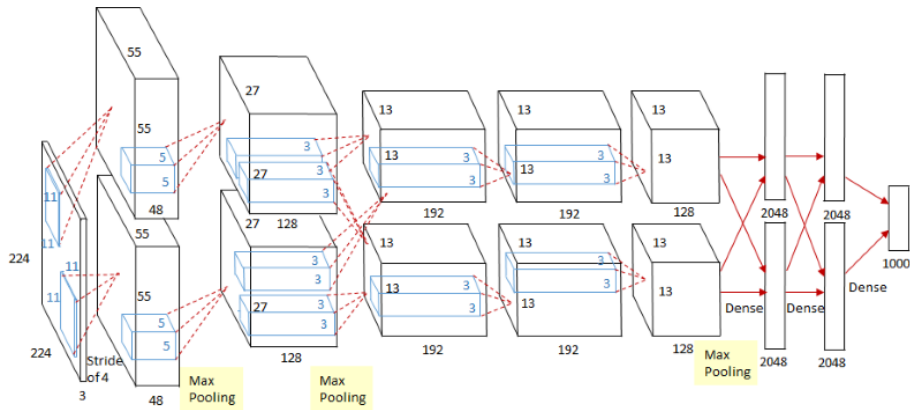
Human top-5 error on this task is about 5%
— crossed in 2015.



Winning top-5 error, and depth, by year

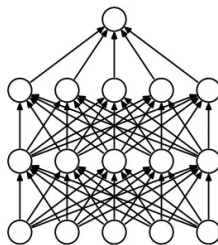
AlexNet (2012)

Very similar architecture to LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other

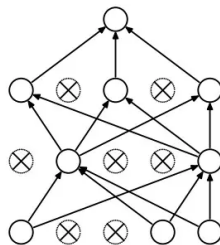


AlexNet's Special Features

- 8 weight layers, **60M parameters**, trained on two GTX 580s for six days
- First large-scale use of **ReLU** instead of tanh and sigmoid — several times faster to converge
- Dropout in the FC layers; aggressive data augmentation
- 11×11 first-layer filters with stride 4 — huge by modern standards



(a) Standard Neural Net



(b) After applying dropout.

Figure: Dropout

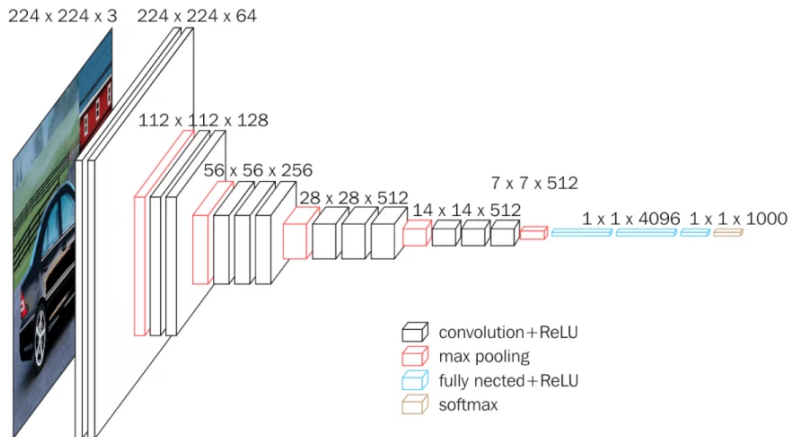
Nothing in it was conceptually new.

What was new: **ImageNet** (enough data) and **GPUs** (enough compute).

The lesson the field took: scale the data and the compute, not just the idea.

VGG (2014): Depth Through Uniformity

- An improvement over AlexNet
- Shows that network depth is an essential element for good performance



VGG's Special Features

- Uniformity: **Only** 3×3 convolutions, stride 1, pad 1, and 2×2 max-pool. Double the channels after every pool.
- VGG-16 and VGG-19; 16 and 19 weight layers
- Three stacked 3×3 layers match one 7×7 receptive field with $27C^2$ parameters instead of $49C^2$ — and two extra nonlinearities
- Limitation: **138M parameters**, most of them in the first FC layer alone

The cost

VGG-16 needs about 15.5 GFLOPs per image — roughly $4\times$ GoogLeNet for worse accuracy. It is a memory hog and was obsolete within a year.

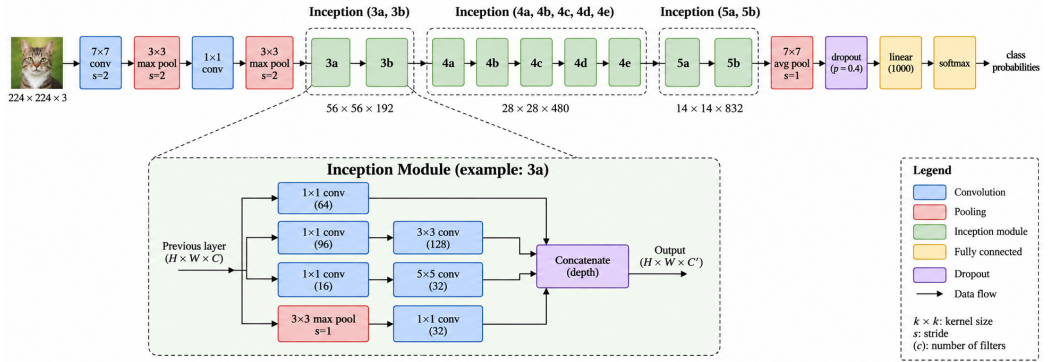
The legacy

The “small filters, uniform blocks, double the channels when you halve the resolution” recipe still underpins almost every backbone.

GoogLeNet (2014): Efficiency

Introduced the Inception layer that dramatically reduced the number of parameters in the network

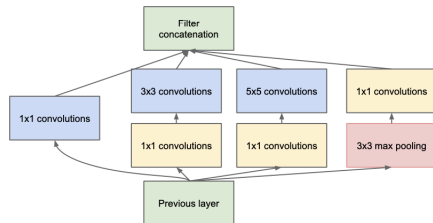
GoogLeNet (Inception-v1)



GoogLeNet's Special Features

22 layers, only **5M parameters** — $12\times$ fewer than AlexNet, while winning.

- **Inception module:** run 1×1 , 3×3 , 5×5 convolutions and pooling *in parallel*, concatenate the outputs. Let the network choose the scale
- 1×1 **bottlenecks** before the expensive branches, cutting channel depth first — this is what makes it affordable
- **No large FC layers:** global average pooling instead
- Auxiliary classifiers midway through, to push gradient into early layers



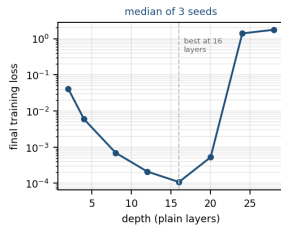
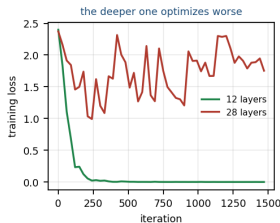
An Inception module with 1×1 reductions

The Degradation Problem

Take a working 20-layer network. Make it 56 layers. It gets **worse** — on the *training* set.

- Not overfitting: the training error is higher too
- Not vanishing gradients: BatchNorm already fixed that
- The deeper network *contains* the shallower one as a special case — copy the 20 layers, make the other 36 the identity — so it can never be worse in principle

So the problem is optimization: SGD cannot find that solution. Learning an identity mapping through a stack of conv layers is apparently hard.



Deeper plain network, worse training error

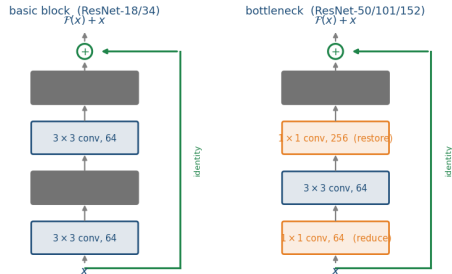
ResNet (2015): Learn the Residual

Developed by Kaiming He et al. to solve both the vanishing gradient and degradation problems. If the identity is hard to learn, **build it in** and learn only the difference:

$$y = \mathcal{F}(x, \{W_i\}) + x$$

- The block only has to produce the *residual* $\mathcal{F}$
- To recover the identity, drive the weights to zero, which weight decay already does
- The addition is parameter-free
- Gradients reach earlier layers through the skip path unchanged: $\partial y / \partial x = \partial \mathcal{F} / \partial x + 1$

With $+1$, the block can never fully kill the gradient.



Basic block and bottleneck block

- **ResNet-18/34**: two 3×3 layers per block
- **ResNet-50/101/152**: *bottleneck* blocks — 1×1 reduce, 3×3 convolve, 1×1 restore. Cheaper and deeper
- Every conv followed by BatchNorm; no FC layers except the classifier; global average pooling before it
- When the shape changes, the skip path uses a 1×1 stride-2 convolution
- ResNet-152: 3.57% top-5, and **fewer parameters than VGG-16**

Why this mattered beyond vision

Residual connections turned out to be the general answer to depth. They are in every transformer block, every diffusion U-Net, every large language model you have used.

After ResNet: Four Directions

Residual connections settled the depth question. The next few years asked different ones.

Better connectivity — DenseNet (2017)

Every layer takes *all* previous layers as input, **concatenated** rather than added. Maximum feature reuse; DenseNet-121 matches ResNet-50 with $\sim 8\text{M}$ parameters instead of 25M. Memory-hungry in exchange.

Recalibration — SENet (2017)

Squeeze-and-excitation: pool each channel to one number, learn a small gate, rescale the channels. The network decides which feature maps matter *for this image*. Won the final ILSVRC at 2.25%.

More paths — ResNeXt (2017)

Replace one wide convolution with G parallel grouped ones. A third axis besides depth and width, which the authors call **cardinality** — and it buys more accuracy per parameter than either.

Efficiency — MobileNet (2017)

Built for phones: depthwise separable convolutions, $\sim 4\text{M}$ parameters. The next slide is how it works.

The Architectures Side by Side

Model	Year	Layers	Params	Top-5	Key idea
AlexNet	2012	8	60M	15.3%	ReLU, dropout, GPUs
VGG-16	2014	16	138M	7.3%	Only 3×3 , uniform blocks
GoogLeNet	2014	22	5M	6.7%	Inception, 1×1 bottlenecks
ResNet-50	2015	50	25M	~5%	Residual connections
ResNet-152	2015	152	60M	3.57%	Residual connections

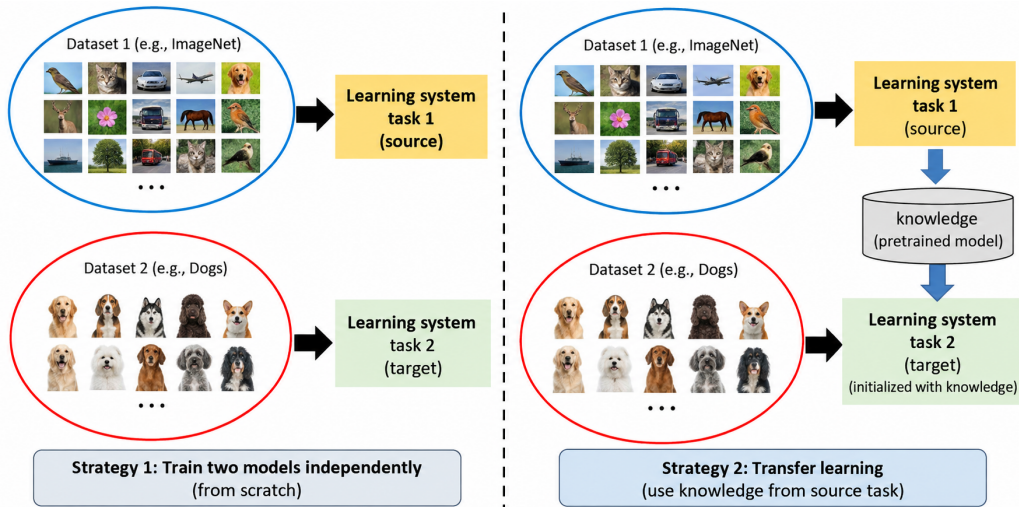
Accuracy went up by a factor of four while parameters went *down*. Depth and structure beat raw size.

What to use in your project

ResNet-50 unless you have a reason. It is the default baseline in almost every vision paper, and every framework ships pretrained weights for it.

Transfer Learning

Traditional ML vs. Transfer Learning



Why Not Training From Scratch?

ImageNet training takes days on many GPUs and 1.28M labeled images. Your project has one GPU and perhaps a few thousand images.

But the early layers of any trained CNN learn edges, colors, and textures — features that are not specific to ImageNet's thousand classes, and that your task needs too.

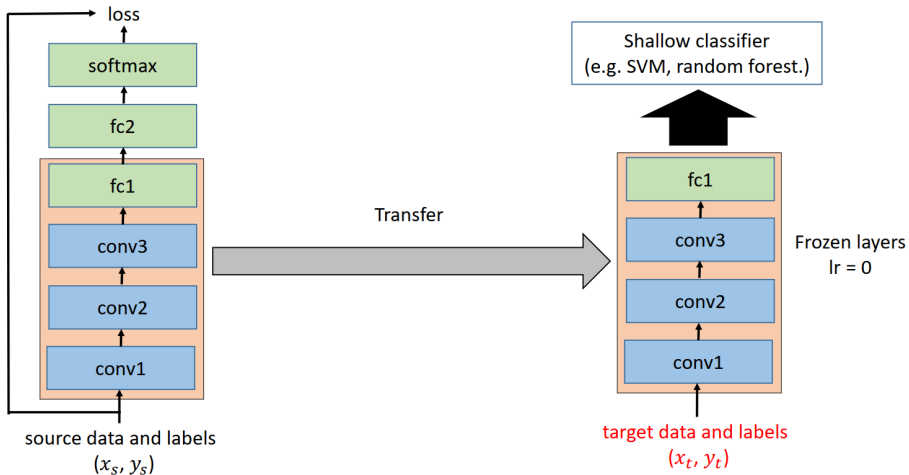
Using a pretrained network is not a shortcut; it is the standard method, and it usually *beats* training from scratch on small data.

Two modes

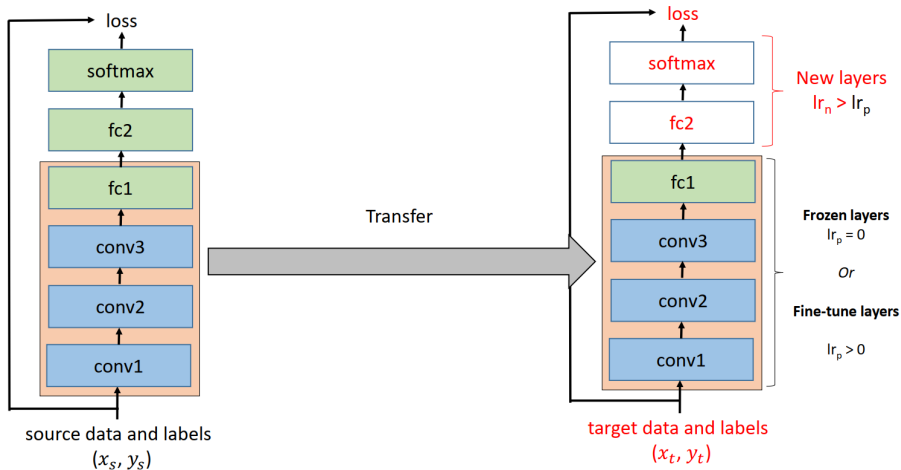
Feature extraction — freeze the backbone, replace and train only the final classifier. Fast, few parameters, little overfitting risk.

Fine-tuning — also update some or all of the backbone, at a **much smaller** learning rate ($10\times$ to $100\times$ smaller).

Feature Extraction



Fine-Tuning



??

Domain

Two domains are different if they have different feature spaces or different marginal distributions

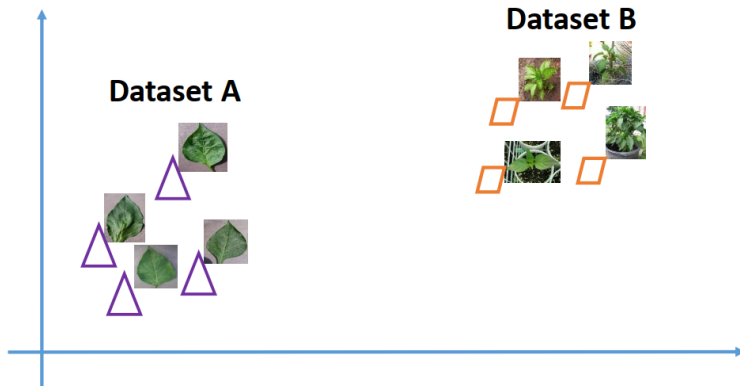
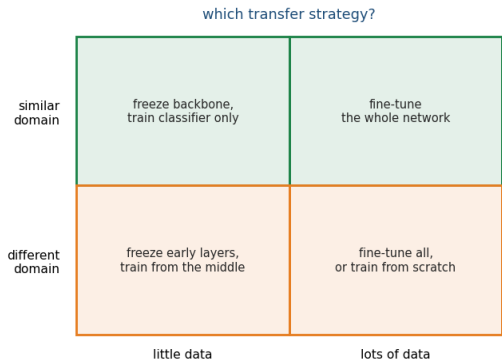


Figure: Bell pepper data

Which Strategy?



Data size × domain similarity

- **Small data, similar domain:** freeze everything, train the classifier only. Fine-tuning would overfit immediately
- **Large data, similar domain:** fine-tune the whole network — the safest, best-performing case
- **Small data, different domain:** the hard case. Freeze early layers, train from an intermediate layer; expect to work for your accuracy
- **Large data, different domain:** fine-tune everything, or consider training from scratch

A Practical Recipe

```
from torchvision import models

m = models.resnet50(weights="DEFAULT")

# 1. freeze the backbone
for p in m.parameters():
    p.requires_grad = False

# 2. new head for YOUR classes
m.fc = nn.Linear(2048, num_classes)

# 3. train the head only
opt = torch.optim.Adam(
    m.fc.parameters(), lr=1e-3)
```

Then, if you have the data:

- 1 Unfreeze the last block or two
- 2 Drop the learning rate to 10^{-4} or below
- 3 Continue training

Two details that bite

Use the **same normalization** the pretrained model was trained with (ImageNet mean and std). And keep BatchNorm layers in eval mode while the backbone is frozen, or their running statistics will drift.

Wrap-Up

Summary

- **Initialization** keeps the signal alive at step 0: $\text{Var}(w) = 2/n_{\text{in}}$ for ReLU, because ReLU discards half the variance
- **BatchNorm** keeps it alive during training, permits higher learning rates, and behaves differently at test time — remember `model.eval()`
- **Momentum** damps oscillation and accelerates along valleys; **Adam** adds a per-parameter learning rate; **schedules** decay η as you converge
- **Augmentation** is domain knowledge written as code, and the strongest regularizer in vision
- **Architecture**: bigger (AlexNet), deeper and uniform (VGG), cheaper (GoogLeNet), and then **residual** (ResNet) — which made depth itself trainable
- **Transfer learning**: your project will do. ResNet-50, pretrained, fine-tuned

Next week: vision transformers — attention replaces convolution.

Questions?

- Krizhevsky, Sutskever & Hinton, “ImageNet Classification with Deep CNNs”, *NeurIPS* 2012
- Simonyan & Zisserman, “Very Deep Convolutional Networks” (VGG), *ICLR* 2015
- Szegedy et al., “Going Deeper with Convolutions” (GoogLeNet), *CVPR* 2015
- He, Zhang, Ren & Sun, “Deep Residual Learning for Image Recognition”, *CVPR* 2016
- He et al., “Delving Deep into Rectifiers” (He initialization), *ICCV* 2015
- Ioffe & Szegedy, “Batch Normalization”, *ICML* 2015; Santurkar et al., *NeurIPS* 2018

Notebook: `lecture05_handson.ipynb`

- Measure activation collapse: the same deep network at four initialization scales
- Verify the $2/n_{in}$ rule numerically, then break it and watch the signal die
- Train with and without BatchNorm at a learning rate that only one of them survives
- Reproduce the `model.eval()` bug on purpose, and see the accuracy change
- Compare SGD, momentum, RMSProp, and Adam on the same problem
- Build an augmentation pipeline and inspect what the model actually receives
- Fine-tune a pretrained ResNet-18 on a small dataset — frozen vs. unfrozen